

SQL 조회, 함수, 그룹 요약

오늘의 강의 학습 요약

본 강의에서는 SELECT 문의 기본 구조와 결과 컬럼 헤더가 결정되는 원리(표현식/별칭), NULL의 의미와 연산, 비교 시 주의점, 그리고 제약조건을 통한 무결성 보장 개념을 정리했습니다. 이어서 WHERE 조건절의 비교, 범위, 집합, 패턴, 논리 연산 및 ORDER BY 정렬 규칙(컬럼/별칭/순서 사용)을 체계적으로 묶어 설명했습니다.

이후 본격적으로 함수로 확장하여 단일행 함수(문자/수치/날짜/기타)와 그룹 함수의 차이를 "적용 단위(행 vs 그룹)"로 구분했습니다. 문자열 함수(대소문자, 연결, 패딩, 부분열, 길이, 치환, 위치, 공백/문자 제거, 반복, 역순, 공백 생성, 포맷), 수치 함수(절댓값, 부호, 반올림, 절사, 올림/내림, 나머지), 날짜 함수(현재 시각, 날짜/시간 분리, 더하기/빼기, 차이, 월말, 단위 추출, 포맷 변환)를 다뤘고, 조건 함수(IF/CASE), 형변환(CAST), 마지막으로 그룹 함수와 GROUP BY/HAVING 규칙 및 COUNT의 NULL 처리까지 정리했습니다.

1. SELECT 기본과 NULL/제약조건

SELECT는 컬럼 선택, 레코드 필터링, 조인 기반 조회를 수행하며, 결과 컬럼 헤더는 SELECT절 표현식에 의해 결정되고 별칭으로 제어할 수 있습니다.

SELECT 문의 기본 골격은 SELECT 절 + FROM 절이며, SELECT 절에는 컬럼명뿐 아니라 *, 리터럴, 산술 연산, 함수 호출 같은 "표현식"이 올 수 있습니다. SELECT 절에 작성한 표현식은 그대로 결과 테이블의 컬럼 헤더로 노출되므로, 사람이 읽기 좋은 헤더가 필요하다면 별칭(alias)을 사용해 이름을 바꿉니다.

별칭은 AS 별칭 형태이며 AS 는 생략 가능하지만, ANSI 호환성과 가독성을 위해 AS 를 명시하는 습관이 권장됩니다. 별칭에 공백 등 특수한 형태가 필요하다면 쌍따옴표를 사용할 수 있습니다.

SELECT 실행 결과는 기본적으로 "전체 레코드"를 대상으로 하며, 필요한 레코드만 가져오려면 이후에 WHERE 절을 추가해야 합니다. 또한 ANSI 표준은 SELECT ... FROM ... 형태가 기본이지만, MySQL은 FROM 절 생략이 가능한 경우가 있으나(예: 상수 조회) 호환성 측면에서 기본 문법을 고수하는 것이 안전합니다.

NULL은 "비어있는 값"이며, 대부분의 컬럼은 기본적으로 NULL을 가질 수 있습니다. NULL을 허용하지 않게 하려면 NOT NULL 제약조건으로 강제합니다.

NULL은 연산 시 중요한 성질을 갖습니다. NULL을 포함해 산술 연산을 수행하면 결과도 NULL이 되며, 이를 방지하려면 NULL을 기본값으로 치환하는 함수(예: IFNULL(컬럼, 기본값))를 사용합니다. 또한 NULL 비교는 = 로 할 수 없고 IS NULL , IS NOT NULL 을 사용해야 합니다.

제약조건(Constraint)은 저장되는 데이터의 "결점 없음"을 강제하여 무결성을 보장합니다. ANSI 표준 제약조건으로는 주요하게 PK, UNIQUE, CHECK, FK, NOT NULL을 다루며, 각각의 목적과 차이를 명확히 구분해야 합니다.

제약조건	핵심 목적	NULL 허용	특징 요약
PRIMARY KEY	레코드 식별(유일 +필수)	불가	테이블당 1개이며, NOT NULL + UNIQUE 조합이고 인덱스가 자동 생성됩니다.
UNIQUE	값의 유일성 보장	가능	여러 개 지정 가능하며, 인덱스가 자동 생성됩니다.
CHECK	비즈니스 규칙 검증	상황에 따름	성별(F/M), 학년 범위 등 저장 규칙을 DB에서 강제할 수 있습니다.
FOREIGN KEY	참조 무결성 보장	가능	참조 대상은 마스터 테이블의 PK 또는 UNIQUE 컬럼이어야 합니다.
NOT NULL	NULL 저장 금지	불가	컬럼 레벨로만 지정되는 "수정 성격"의 제약으로 이해하면 정리됩니다.

FOREIGN KEY는 두 테이블이 관여하므로 정의 구문이 길어지는 경향이 있으며, 조인 관계를 데이터 수준에서 강제하여 “존재하지 않는 부서번호” 같은 결함 데이터를 원천 차단합니다. FK 컬럼이 가질 수 있는 값은 참조 대상의 PK/UNIQUE 값 또는 NULL(허용 시)로 정리됩니다.

PRIMARY KEY는 레코드 식별에 사용되며, 단일 컬럼 또는 복합 컬럼(2~3개 컬럼 묶음)으로 정의할 수 있습니다. 또한 PK/UNIQUE는 인덱스 자동 생성으로 검색 성능에 유리합니다.

핵심 키워드

#SELECT절

#별칭 (AS)

#NULL

#제약 조건

#무결성

2. WHERE 조건식과 정렬(ORDER BY)

WHERE는 전체 레코드 중 필요한 행만 필터링하며, 비교/범위/집합/패턴/논리 연산을 조합하고 ORDER BY로 정렬 기준을 제어합니다.

WHERE 절은 전체 레코드에서 조건을 만족하는 레코드만 추출합니다. SQL에서 동등 비교는 자바의 `==` 가 아니라 `=` 를 사용하며, 문자열, 날짜 리터럴은 ANSI 호환을 위해 작은따옴표를 사용합니다.

MySQL은 문자열 비교에서 대소문자를 엄격히 구분하지 않는 설정이 일반적이지만(콜레이션 영향), Oracle은 대소문자 구분이 중요한 경우가 많습니다. 이식성을 고려하면 대소문자 처리(UPPER/LOWER 등)를 의도적으로 설계하는 편이 안전합니다.

범위 조건은 `BETWEEN A AND B` 를 사용하며 A, B를 포함하는 inclusive 범위입니다. 날짜도 내부적으로 비교 가능한 값으로 관리되므로 비교/범위 연산을 적용할 수 있습니다.

집합 조건은 `IN (...)` 을 사용하며 내부적으로 OR 조건의 축약으로 이해할 수 있습니다. 패턴 검색은 `LIKE` 와 와일드카드(`%`, `_`)를 사용하며, `%` 는 0개 이상 문자, `_` 는 정확히 1문자를 치환합니다.

논리 연산은 `AND`, `OR`, `NOT` 으로 조건을 결합하거나 부정합니다. 특히 NULL은 `LIKE` 나 `=` 로 찾을 수 없고 `IS NULL` 로만 검색해야 합니다.

정렬은 `ORDER BY` 로 수행하며 기본은 오름차순(ASC)이고 내림차순은 DESC를 지정합니다. ORDER BY는 문장(SELECT 구문의 구성요소) 중 가장 마지막에 위치합니다.

ORDER BY 뒤에는 다음 중 하나를 사용할 수 있으며, 이는 실무에서 자주 사용되는 규칙입니다.

- ORDER BY에는 SELECT 절의 컬럼명을 지정할 수 있습니다.
- ORDER BY에는 SELECT 절에서 부여한 별칭을 지정할 수 있습니다.
- ORDER BY에는 SELECT 절 컬럼의 "순번(1,2,3...)"을 지정할 수 있습니다.
- 다중 정렬은 앞 컬럼으로 1차 정렬 후 값이 같을 때 다음 컬럼으로 2차 정렬합니다.

핵심 키워드

#WHERE

#BETWEEN

#IN

#LIKE

#ORDER BY

3. 단일행 함수: 문자열 함수와 중첩

단일행 함수는 행 단위로 적용되어 입력 행 수만큼 결과가 나오며, 문자열 가공(대소문자, 연결, 패딩, 부분열, 길이, 치환, 공백 처리 등)에 활용됩니다.

함수는 자바의 메소드처럼 데이터를 가공하는 도구이며, SQL에서는 크게 단일행 함수(행마다 적용)와 그룹 함수(그룹마다 적용)로 구분합니다. 단일행 함수는 레코드가 14개면 함수 적용 결과도 14행이 그대로 유지되는 방식입니다.

문자열 대소문자 변환은 `LOWER`, `UPPER` 를 사용합니다. 리터럴에도 적용할 수 있으므로, 출력 형식 통일이나 비교 전 표준화에 유용합니다.

문자열 연결은 `CONCAT(str1, str2, ...)` 로 수행하며 인자 개수는 2개에 제한되지 않습니다. 구분자까지 자동 삽입하며 연결하려면 `CONCAT_WS(separator, str1, str2, ...)` 를 사용합니다.

패딩은 지정 길이까지 특정 문자로 채우는 기능입니다. `LPAD(str, len, pad)` 는 왼쪽 채움, `RPAD(str, len, pad)` 는 오른쪽 채움입니다.

부분열은 `SUBSTR(str, pos, len)` (또는 `SUBSTRING`)로 추출하며, 위치(pos)는 자바처럼 0부터가 아니라 1부터 시작하는 점이 실수 포인트입니다. `len` 을 생략하면 pos부터 끝까지 반환됩니다.

길이 함수는 반드시 "바이트 길이"와 "문자 길이"를 구분해야 합니다. `LENGTH` 는 바이트 수를 반환하므로 한글처럼 멀티바이트 문자는 글자 수와 달라질 수 있습니다. 일반적으로 글자 개수가 필요하다면 `CHAR_LENGTH` 를 사용합니다.

함수	반환 기준	한글 예시("홍길동")에서 의미
<code>LENGTH</code>	바이트 수	UTF-8 환경에서 9가 될 수 있습니다(글자당 3바이트 가정).
<code>CHAR_LENGTH</code>	문자 개수	3으로 계산됩니다.

치환은 `REPLACE(str, from, to)` 로 특정 문자열을 바꾸며, `INSERT(str, pos, len, newstr)` 로 지정 위치의 일정 길이를 다른 문자열로 치환할 수도 있습니다.

문자 위치는 `INSTR(str, substr)` 로 찾으며, 존재하지 않으면 0을 반환합니다. 자바의 `indexOf` 가 미존재 시 -1을 반환하는 것과 다르므로 주의가 필요합니다.

공백 제거는 `LTRIM` (왼쪽), `RTRIM` (오른쪽), `TRIM` (양쪽)이 기본이며, `TRIM`은 공백뿐 아니라 특정 문자 제거까지 확장 문법을 지원합니다.

`TRIM`의 확장 문법은 다음 의미로 정리됩니다.

형태	제거 방향	의미
<code>TRIM(LEADING 'x' FROM str)</code>	왼쪽	왼쪽의 지정 문자를 제거합니다.
<code>TRIM(TRAILING 'x' FROM str)</code>	오른쪽	오른쪽의 지정 문자를 제거합니다.
<code>TRIM(BOTH 'x' FROM str)</code>	양쪽	양쪽의 지정 문자를 제거합니다.

기타 문자열 함수로 `REPEAT(str, count)` (반복), `REVERSE(str)` (역순), `SPACE(n)` (공백 n개 생성)도 사용할 수 있습니다.

함수는 중첩 호출이 가능하며, 안쪽 함수부터 평가됩니다. 다만 바깥 함수가 요구하는 데이터 타입과 안쪽 함수의 반환 타입이 호환되어야 합니다. 예를 들어 `CHAR_LENGTH(TRIM(str))` 는 가능하지만, 문자열이 필요한 자리에 숫자 반환 함수를 끼우면 타입 불일치 오류가 발생할 수 있습니다.

핵심 키워드

#LOWER/UPPER

#CONCAT

#LPAD/RPAD

#SUBSTR

#CHAR_LENGTH

4. 수치 함수와 나머지(MOD) 권장

수치 함수는 절댓값, 부호, 반올림, 절사, 올림/내림, 나머지 등을 제공하며, 나머지는 이식성을 위해 MOD 함수를 권장합니다.

수치 함수는 숫자 데이터를 가공하는 단일행 함수입니다. `ABS(x)` 는 절댓값을 반환합니다.

부호는 `SIGN(x)` 로 판별하며, 음수는 -1, 0은 0, 양수는 1을 반환합니다. 값의 크기와 무관하게 부호만 알려주는 용도로 사용합니다.

반올림은 `ROUND(x[, d])` 이며 `d` (자릿수)를 생략하면 정수로 반올림됩니다. 절사(버림)는 `TRUNCATE(x, d)` 로 수행하며, `ROUND`와 달리 자릿수 `d` 를 반드시 지정해야 합니다.

올림/내림은 `CEIL` (또는 `CEILING`)과 `FLOOR` 로 처리합니다. `CEIL`은 “값보다 크거나 같은 최소 정수”, `FLOOR`는 “값보다 작거나 같은 최대 정수”라는 정의로 이해해야 음수에서 헛갈리지 않습니다.

함수	1.23	-1.23	의미
<code>CEIL</code>	2	-1	값 이상 중 최소 정수입니다.
<code>FLOOR</code>	1	-2	값 이하 중 최대 정수입니다.

나머지는 `%` 연산자 또는 `MOD(a, b)` 로 구할 수 있습니다. 다만 Oracle은 `%` 연산자를 지원하지 않고(문맥에 따라 `%` 가 LIKE 와일드카드로 해석되는 문제), 이식성을 고려하면 MOD 함수를 사용하는 것이 권장됩니다.

핵심 키워드

#ABS

#SIGN

#ROUND

#TRUNCATE

#MOD

5. 날짜 함수와 포맷 변환

날짜는 현재 시각 조회, 날짜/시간 분리, 더하기, 빼기, 차이 계산, 월말 계산, 단위 추출, 문자열↔날짜 변환으로 다루며, 일부는 DB별 비호환이 존재합니다.

MySQL 날짜 함수는 ANSI가 아닌 것들이 섞여 있어 DB 이식성에 주의가 필요합니다. 현재 날짜+시간은 `NOW()` 로 얻으며 기본 출력 포맷은 `YYYY-MM-DD HH:MM:SS` 입니다. `NOW()+0` 처럼 0을 더하면 하이픈 없이 숫자 형태로 표현되는 예시가 문서에 제시됩니다.

`SYSDATE()` 도 유사하게 현재 시각을 반환하지만, 동작 시점이 다릅니다. `NOW()` 는 “SQL 문이 시작된 시점”을 기준으로 동일 문장 내 값이 일관되게 유지되는 반면, `SYSDATE()` 는 “함수를 호출한 시점” 기준이라 동일 문장 내에서도 시간이 달라질 수 있습니다(예: `SLEEP(2)` 사이).

현재 날짜만은 `CURDATE()` 이며, 동의어로 `CURRENT_DATE` , `CURRENT_DATE()` 형태도 사용할 수 있습니다. 현재 시간만은 `CURTIME()` 이며, 동의어로 `CURRENT_TIME` , `CURRENT_TIME()` 를 사용할 수 있습니다. 또한 `NOW()` 의 동의어로 `CURRENT_TIMESTAMP` , `CURRENT_TIMESTAMP()` 도 정리됩니다.

날짜 더하기/빼기는 `ADDDATE(date, days)` , `SUBDATE(date, days)` 로 처리할 수 있으며, 동의어 (`DATE_ADD` , `DATE_SUB`)도 존재합니다. 더 일반적으로는 `INTERVAL` 과 `unit` 을 사용해 연/월/일/시/분 등 단위를 지정할 수 있습니다.

목적	예시 형태	의미
단위 기반 더하기	<code>DATE_ADD(d, INTERVAL n UNIT)</code>	UNIT에 YEAR, MONTH, DAY, HOUR, MINUTE 등을 사용합니다.
단위 기반 빼기	<code>DATE_SUB(d, INTERVAL n UNIT)</code>	UNIT별로 동일하게 감산할 수 있습니다.

날짜 차이는 `DATEDIFF(date1, date2)` 로 “일(day) 단위” 차이를 반환합니다. 단위를 지정해 차이를 구하려면 `TIMESTAMPDIFF(UNIT, datetime1, datetime2)` 를 사용하며, UNIT으로 YEAR/MONTH/DAY/HOUR/MINUTE 등을 지정할 수 있습니다.

특정 월의 마지막 날은 `LAST_DAY(date)` 로 구합니다. 또한 날짜에서 연/월/일/시/분/초 등 특정 단위만 추출하는 함수로 `EXTRACT(UNIT FROM date)` 가 있으며, 이는 오라클에서도 사용 가능한 ANSI 성격으로 함께 정리되었습니다.

날짜를 문자열로 바꾸는 것은 `DATE_FORMAT(date, format)` 이며, 지정 포맷에 따라 원하는 출력 문자열을 생성합니다. 포맷 토큰은 예를 들어 연도 `%Y` , 월 `%m` , 일 `%d` , 24시간 `%H` , 분 `%i` , 초 `%s` 등으로 구성됩니다.

문자열을 날짜로 바꾸는 것은 `STR_TO_DATE(str, format)` 이며, 이때 `format`은 “출력 포맷”이 아니라 입력 문자열의 모양을 DB가 날짜로 해석할 수 있도록 안내하는 포맷입니다. 입력 문자열의 구분자(하이픈, 슬래시, 공백 등)와 토큰 구조가 `format`과 정확히 일치해야 하며, 불일치 시 오류가 발생합니다.

핵심 키워드

`#NOW/SYSDATE`

`#DATE_ADD`

`#DATEDIFF`

`#DATE_FORMAT`

`#STR_TO_DATE`

6. 조건 함수(IF/CASE)와 형변환(CAST)

IF는 단일 조건 분기, CASE는 다중 분기(동등/부등 형태)를 제공하며, CAST로 데이터 타입을 명시적으로 변환할 수 있습니다.

조건 함수는 분기 로직을 SQL 결과에 반영할 때 사용합니다. IF(condition, expr_true, expr_false)는 조건이 참이면 두 번째 표현식을, 거짓이면 세 번째 표현식을 반환합니다. 예를 들어 급여가 3000 이상이면 "과장급 이상", 아니면 "일반 사원"처럼 결과 컬럼을 만들 수 있습니다.

CASE는 두 가지 형태로 구분됩니다. 하나는 스위치문처럼 "동등 비교"를 수행하는 형태이며, 다른 하나는 "조건식(부등 포함)"을 WHEN 절에 직접 작성하는 형태입니다.

형태	문법 핵심	비교 방식
동등 비교 CASE	CASE value WHEN compare THEN ... ELSE ... END	내부적으로 = 기반 동등 비교에 가깝습니다.
조건식 CASE	CASE WHEN condition THEN ... ELSE ... END	크다/작다 등 부등 조건을 직접 작성합니다.

CASE는 SELECT 절의 "표현식"으로 들어가며, 결과 컬럼 헤더가 길어지므로 보통 별칭을 함께 부여합니다. 예시로 JOB 별로 급여에 가중치(1.1/1.2/1.3/1.4)를 곱하고, 나머지는 원 급여를 출력하는 로직을 CASE로 구현할 수 있습니다. 또한 급여 구간에 따라 직급 문자열을 만들 때는 조건식 CASE 형태가 적합합니다.

형변환은 CAST(expr AS type) 또는 CONVERT 계열을 통해 수행합니다. 실무에서는 숫자↔문자, 문자↔숫자 같은 변환을 자주 사용합니다. MySQL 타입으로는 CHAR, FLOAT, UNSIGNED INTEGER 등 다양한 타입을 지정할 수 있으며, 부호 있는 정수(SIGNED)와 부호 없는 정수(UNSIGNED)도 구분됩니다.

날짜 변환도 CAST로 시도할 수 있으나, 문자열 날짜 파싱은 보통 STR_TO_DATE가 입력 포맷을 명확히 지정할 수 있어 더 적합하다는 점을 함께 정리할 수 있습니다.

핵심 키워드

#IF

#CASE

#동등비교 CASE

#조건식 CASE

#CAST

7. 그룹 함수와 GROUP BY/HAVING 규칙

그룹 함수는 그룹 단위로 평균, 합계, 최대, 최소, 개수를 계산하며, GROUP BY로 그룹을 만들고 HAVING으로 그룹 결과를 필터링합니다.

그룹 함수(집계 함수)는 행 단위가 아니라 “그룹 단위”로 적용됩니다. 따라서 원본 레코드가 14개여도 부서별로 3그룹이면 결과는 3행만 나오며, 잡별로 5그룹이면 5행이 나옵니다. 그룹을 명시적으로 만드는 문법이 GROUP BY 입니다.

자주 쓰는 그룹 함수는 다음으로 정리됩니다.

함수	의미	기본 형태
SUM(expr)	합계	지정 컬럼(표현식)의 합계를 구합니다.
AVG(expr)	평균	지정 컬럼(표현식)의 평균을 구합니다.
MAX(expr)	최댓값	지정 컬럼(표현식)의 최댓값을 구합니다.
MIN(expr)	최솟값	지정 컬럼(표현식)의 최솟값을 구합니다.
COUNT(expr) / COUNT(*)	개수	expr은 NULL 제외, * 는 전체 레코드 수입니다.

GROUP BY 뒤에는 MySQL에서 컬럼명, 별칭, 순서(위치)를 사용할 수 있어 ORDER BY 와 유사한 편의가 있습니다. 다만 이 중 일부(특히 별칭/순서)는 DBMS별 지원이 다를 수 있으므로, 이식성을 고려하면 컬럼명 기반 사용이 안전합니다.

HAVING은 그룹 결과에 조건을 적용하는 절입니다. 처리 순서는 “FROM → WHERE(행 필터링) → GROUP BY(그룹핑) → HAVING(그룹 필터링) → ORDER BY(정렬)”로 이해하면, WHERE에 그룹 함수를 쓸 수 없는 이유가 자연스럽게 설명됩니다.

- WHERE는 그룹핑 이전 단계이므로 집계 결과(SUM/AVG 등)를 사용할 수 없습니다.
- HAVING은 그룹핑 이후 단계이므로 집계 결과에 조건을 걸 수 있습니다.

또한 SELECT 절에 그룹 함수와 함께 “일반 컬럼”을 출력하려면, 그 일반 컬럼은 반드시 GROUP BY에 포함되어야 합니다. 예를 들어 부서별 집계를 수행하면서 JOB 컬럼을 함께 SELECT에 넣으면, JOB이 그룹 키가 아니므로 결과 행 수가 맞지 않아 오류가 발생합니다. 이를 해결하려면 GROUP BY deptno, job 처럼 함께 그룹핑해야 합니다.

마지막으로 COUNT는 NULL 처리 규칙이 매우 중요합니다. COUNT(commission) 은 commission이 NULL 인 행을 제외하고 세므로, 커미션이 있는 세일즈맨만 카운트되는 식의 결과가 나올 수 있습니다. 전체 레코드 개수가 목적이면 COUNT(*) 를 사용해야 의도와 일치합니다.

사용 목적	권장 형태	이유
전체 레코드 수	COUNT(*)	NULL 여부와 무관하게 모든 행을 셉니다.
특정 컬럼 값이 존재하는 행 수	COUNT(column)	NULL을 제외하고 카운트합니다.

핵심 키워드 #GROUP BY #HAVING #집계함수 #COUNT(*) #NULL 제외 카운트